

CSS MEDIA QUERIES

Complete Detailed Notes • Concepts + Examples + Exam Revision

What you will learn

Media types • media features • responsive design • breakpoints • logical operators • orientation • print styles • accessibility preferences • practical HTML/CSS examples • container queries • common mistakes

Core idea: A media query applies CSS only when a specified condition is true.

Quick memory: “WHEN this condition is true → APPLY these CSS rules.”

1. What is a Media Query?

A **media query** is a CSS feature that lets you apply CSS rules conditionally based on characteristics of the device, viewport, output medium, or user preferences. Media queries are one of the main tools used for **responsive web design**.

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

The rule applies when the viewport is **600px wide or narrower**.

2. Why Media Queries Are Used

- **Responsive design** — adapt a page to different screen sizes.
- **Layout changes** — change the number of columns or stack content.
- **Typography** — adjust font size and spacing.
- **Navigation** — adapt menus for smaller screens.
- **Printing** — create a clean printer-friendly page.
- **Accessibility** — respond to preferences such as reduced motion or dark mode.

3. Basic Syntax

```
@media media-type and (condition) {  
  selector {  
    property: value;  
  }  
}
```

Media type identifies the medium. **Condition** tests a feature. **Declarations** are applied if the query matches.

4. Media Types

Media type	Meaning	Typical use
all	Applies to all media types.	General/default styling.
screen	Visual screen output.	Phones, tablets, laptops, monitors.
print	Printed or print-preview output.	Printer-friendly CSS.
speech	Speech output.	Speech-oriented user agents.

Important: screen and print are especially useful in practice. speech has limited modern practical use; accessibility should primarily use semantic HTML and appropriate accessibility techniques.

5. all Media Type

all means the styles are intended for all media types. It is normally unnecessary to write it explicitly because general CSS already applies broadly.

```
@media all {  
  body {  
    font-family: Arial, sans-serif;  
  }  
}
```

In normal projects, you will usually write the CSS directly instead of wrapping it in `@media all`.

6. screen Media Type

screen targets visual screen output.

```
@media screen and (max-width: 600px) {  
  .container {  
    width: 100%;  
  }  
}
```

Modern responsive CSS often omits screen: @media (max-width: 600px).

7. print Media Type

Use @media print for printed output and print preview.

```
@media print {  
  .navbar,  
  .advertisement,  
  .buttons {  
    display: none;  
  }  
  
  body {  
    color: black;  
    background: white;  
  }  
}
```

Common print changes include hiding navigation/ads, removing decorative backgrounds, improving readability, and controlling page breaks.

8. speech Media Type

The speech media type is intended for speech output. Older speech-related CSS properties appear in specifications and legacy examples, but support and practical use are limited.

Accessibility rule: do not depend on @media speech to make a site accessible. Use semantic HTML, correct headings, labels, keyboard support, meaningful links, and ARIA where appropriate.

9. max-width and min-width

```
/* Up to 600px */  
@media (max-width: 600px) {  
  .box {  
    font-size: 14px;  
  }  
}  
  
/* 768px and above */  
@media (min-width: 768px) {  
  .box {  
    font-size: 18px;  
  }  
}
```

max-width → matches at or below the specified width.

min-width → matches at or above the specified width.

10. Common Media Features

Feature	Purpose	Example
width	Viewport width	(min-width: 768px)

Feature	Purpose	Example
height	Viewport height	(min-height: 600px)
orientation	Portrait or landscape	(orientation: landscape)
aspect-ratio	Viewport width/height ratio	(aspect-ratio: 16/9)
resolution	Output resolution	(min-resolution: 2dppx)
hover	Whether primary input can hover	(hover: hover)
pointer	Pointer accuracy	(pointer: fine)
prefers-reduced-motion	Motion preference	(prefers-reduced-motion: reduce)
prefers-color-scheme	Light/dark preference	(prefers-color-scheme: dark)
forced-colors	Forced-color mode	(forced-colors: active)

11. Orientation

```
@media (orientation: portrait) {
  .layout {
    display: block;
  }
}

@media (orientation: landscape) {
  .layout {
    display: flex;
  }
}
```

Portrait → taller than wide. **Landscape** → wider than tall.

12. Combining Conditions with and

```
@media (min-width: 768px) and (max-width: 1024px) {
  .card {
    width: 45%;
  }
}
```

Both conditions must be true. The example targets a width range from 768px through 1024px.

13. Logical Operators: and, comma, not, only

```
/* AND – every condition must match */
@media screen and (min-width: 700px) {
  /* styles */
}

/* COMMA – OR-like behavior */
@media (max-width: 600px), (orientation: portrait) {
  /* styles */
}

/* NOT – excludes the specified media query */
@media not print {
  /* styles */
}

/* ONLY – legacy/compatibility-oriented syntax */
@media only screen and (max-width: 600px) {
  /* styles */
}
```

A comma separates independent media queries. If **any one** of them matches, the overall query matches.

14. Breakpoints

A **breakpoint** is a point at which the layout changes. There is no universal set of correct breakpoints. Choose breakpoints based on when the content starts looking cramped or too wide.

Common label	Example range
Small mobile	up to ~480px
Mobile	up to ~600px
Tablet	~601–900px
Laptop/Desktop	~901px+

Best practice: breakpoints should be content-driven, not based only on device names.

15. Mobile-First Design

Mobile-first means the base CSS is designed for smaller screens, then larger layouts are added using min-width.

```
.container {  
  display: block;  
}  
  
@media (min-width: 768px) {  
  .container {  
    display: flex;  
  }  
}
```

Advantages: simpler base styles, progressive enhancement, and a clear responsive workflow.

16. Desktop-First Design

```
.container {  
  display: flex;  
}  
  
@media (max-width: 767px) {  
  .container {  
    display: block;  
  }  
}
```

Desktop-first starts with the large-screen layout and modifies it for smaller screens. Both approaches are valid.

17. Responsive Grid Example

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 16px;  
}  
  
@media (min-width: 600px) {  
  .cards {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (min-width: 1000px) {  
  .cards {  
    grid-template-columns: repeat(4, 1fr);  
  }  
}
```

Result: **1 column** on small screens → **2 columns** on medium screens → **4 columns** on large screens.

18. prefers-color-scheme

This media feature lets CSS respond to the user's preferred color scheme.

```
@media (prefers-color-scheme: dark) {  
  body {  
    background: #111;  
    color: white;  
  }  
}  
  
@media (prefers-color-scheme: light) {  
  body {  
    background: white;  
    color: #111;  
  }  
}
```

19. prefers-reduced-motion

This feature detects when the user requests reduced motion. It is useful for accessibility.

```
@media (prefers-reduced-motion: reduce) {  
  *,  
  *::before,  
  *::after {  
    animation-duration: 0.01ms;  
    animation-iteration-count: 1;  
    transition-duration: 0.01ms;  
    scroll-behavior: auto;  
  }  
}
```

20. Hover and Pointer

```
@media (hover: hover) and (pointer: fine) {  
  .button:hover {  
    transform: translateY(-2px);  
  }  
}
```

hover describes whether the primary input can hover. pointer: fine generally describes a precise pointing device, such as a mouse.

21. Resolution

```
@media (min-resolution: 2dppx) {  
  .logo {  
    /* high-resolution adjustments */  
  }  
}
```

Resolution queries can target high-density output. For responsive images, HTML features such as srcset are often a better tool for choosing image resources.

22. Print Page Breaks

```
@media print {  
  .chapter {  
    break-before: page;  
  }  
  
  .avoid-break {  
    break-inside: avoid;  
  }  
}
```

The modern break-* properties help control how content flows across printed pages.

23. Viewport Meta Tag

For responsive pages, include the viewport meta tag in the HTML <head>:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

width=device-width makes the layout viewport match the device width. **initial-scale=1.0** sets the initial zoom.

24. Media Queries vs Flexbox and Grid

Media queries decide WHEN the design changes. Flexbox and Grid decide HOW elements are laid out. They complement each other.

```
.products {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 800px) {
  .products {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

25. Media Queries vs Container Queries

A media query usually responds to the viewport or device characteristics. A **container query** responds to the available size of an element's containing component.

```
.card-wrapper {
  container-type: inline-size;
}

@container (min-width: 500px) {
  .card {
    display: flex;
  }
}
```

Use media queries for page-level responsive behavior and container queries when a reusable component should react to its own container.

26. Cascade and Media Queries

```
.box {
  color: blue;
}

@media (min-width: 600px) {
  .box {
    color: red;
  }
}
```

When the query matches, its declarations participate in the normal CSS cascade. A media query does **not** automatically make a declaration more important. Specificity, importance, cascade layers, and source order still matter.

27. Common Mistakes

- Confusing max-width and min-width.

- Adding too many arbitrary breakpoints.
- Forgetting the viewport meta tag.
- Assuming every device supports hover.
- Using media queries to hide important content on small screens.
- Using !important unnecessarily to solve cascade problems.
- Relying on speech for accessibility instead of semantic HTML.
- Designing only for one exact device width.

28. Practical Full HTML/CSS Example

This example combines responsive Grid, min-width breakpoints, print CSS, and reduced-motion preferences. The code is kept as a real monospace code block so indentation and symbols remain intact.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

  <style>
    body {
      margin: 0;
      font-family: Arial, sans-serif;
    }

    .nav {
      padding: 20px;
    }

    .content {
      display: grid;
      grid-template-columns: 1fr;
      gap: 20px;
      padding: 20px;
    }

    @media (min-width: 700px) {
      .content {
        grid-template-columns: repeat(2, 1fr);
      }
    }

    @media (min-width: 1000px) {
      .content {
        grid-template-columns: repeat(4, 1fr);
      }
    }

    @media print {
      .nav {
        display: none;
      }

      body {
        color: black;
        background: white;
      }
    }

    @media (prefers-reduced-motion: reduce) {
      *,
      *::before,
      *::after {
        animation: none;
      }
    }
  </style>
</head>
</html>
```

```

        transition: none;
    }
}
</style>
</head>

<body>
    <nav class="nav">Navigation</nav>

    <main class="content">
        <article>Card 1</article>
        <article>Card 2</article>
        <article>Card 3</article>
        <article>Card 4</article>
    </main>
</body>
</html>

```

How it works:

- Below 700px → one column.
- 700px and above → two columns.
- 1000px and above → four columns.
- Printing → navigation disappears and the page becomes printer-friendly.
- Reduced motion → animations and transitions are disabled.

29. Exam-Oriented Quick Revision

- **Media query:** conditionally applies CSS.
- **all:** all media types.
- **screen:** visual screens.
- **print:** printed/print-preview output.
- **speech:** speech output; limited modern practical use.
- **max-width:** up to a specified width.
- **min-width:** from a specified width upward.
- **and:** all conditions must be true.
- **comma:** any comma-separated query can match.
- **orientation:** portrait or landscape.
- **Breakpoint:** point where the layout changes.
- **Mobile-first:** base small-screen styles + min-width enhancements.
- **prefers-reduced-motion:** respond to reduced-motion preference.
- **prefers-color-scheme:** respond to light/dark preference.
- **Container query:** responds to component/container size.

30. One-Line Examples to Memorize

```

@media screen and (max-width: 600px) { ... }

@media print { ... }

@media (min-width: 768px) { ... }

@media (orientation: landscape) { ... }

@media (min-width: 768px) and (max-width: 1024px) { ... }

```

```
@media (prefers-reduced-motion: reduce) { ... }
```

```
@media (prefers-color-scheme: dark) { ... }
```

Final concept: Start with sensible base CSS, then use media queries to change only what needs to change at meaningful conditions. Test the page at multiple widths and orientations.